

UCS645
PARALLEL & DISTRIBUTED COMPUTING



ASSIGNMENT 3:

Submitted by: Sujal Mallick (102303266)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Correlation Assignment

1. Objective

To implement a sequential and parallel version of a correlation computation between multiple input vectors using OpenMP. The experiment evaluates scalability, speedup, efficiency, and performance behavior across different matrix sizes.

2. Problem Description

Given n_y input vectors each containing n_x elements, compute the correlation coefficient between every pair of vectors (i, j) for all $0 \leq j \leq i < n_y$. The result is stored in a lower triangular matrix.

3. Methodology

1. Implemented a sequential baseline using double precision arithmetic.
2. Parallelized outer loop using OpenMP.
3. Evaluated performance for matrix sizes 1000×1000 , 2000×2000 , and 4000×4000 .
4. Measured execution time and computed speedup.

4. Strong Scaling Results

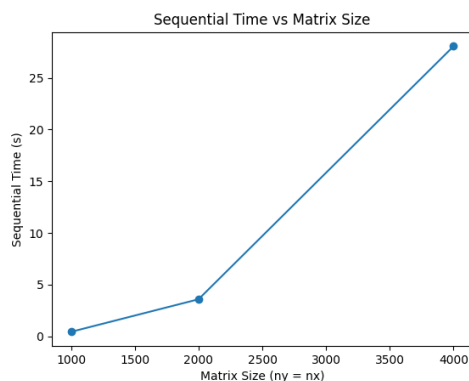
Table 1: Strong Scaling Analysis & Hardware Counters

Matrix Size	Sequential (s)	Parallel(s)	Speedup
1000 x 1000	0.4306	0.2067	2.08x
2000 x 2000	3.5819	1.3530	2.65x
4000 x 4000	28.0567	13.3413	2.10x

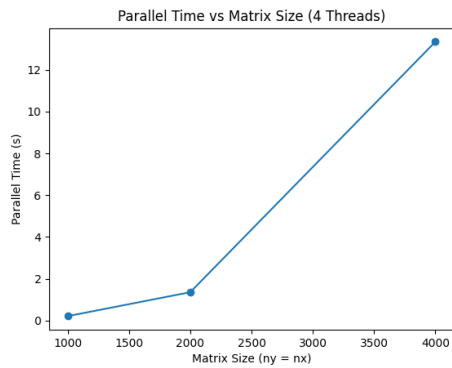
5. Performance Discussion

Execution time increases quadratically with matrix size, as expected from $O(n_y^2 \cdot n_x)$ complexity. Parallel execution significantly reduces runtime. Best performance observed at 2000×2000 with $2.65 \times$ speedup. Speedup is limited by memory bandwidth, cache contention, and triangular loop imbalance. The implementation is partially memory-bound at larger sizes..

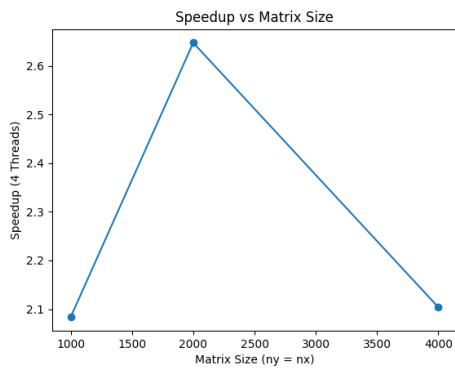
Sequential Time vs Matrix Size



Parallel Time vs Matrix Size (4 Threads)



Speedup vs Matrix Size



7. Conclusion

The OpenMP parallel implementation improves performance compared to the sequential baseline. While ideal linear scaling was not achieved, the program demonstrates effective multi-core utilization and performance gains for moderate matrix sizes.